

Student Analytics Dashboard – Explanation Document

1. Project Overview

This project is a MongoDB-based application that stores large student datasets, performs querying and aggregation, and displays results using Flask and a web UI.

2. Architecture

Frontend (HTML + JS) → Flask Backend → MongoDB (bigdata.students)

3. Database Structure

Database: bigdata

Collection: students

Sample Document:

```
{id, name, sub1..sub6, total, average, grade, dept}
```

4. Functionalities and MongoDB Queries

1. Show All Students (Pagination)

Query: `db.students.find().skip(skip).limit(20)`

2. Top 5 Students

Query: `db.students.find().sort({average: -1}).limit(5)`

3. Average Marks by Grade

Query:

```
db.students.aggregate([
  {$group: {_id: "$grade", avg: {$avg: "$average"}}}
])
```

4. Department-wise Count

Query:

```
db.students.aggregate([
  {$group: {_id: "$dept", count: {$sum: 1}}}
])
```

5. Search by Name

Query:

```
db.students.find({name: {$regex: name, $options: "i"}})
```

6. Filter by Grade

Query:

```
db.students.find({grade: "A"})
```

7. Sort by Average

Ascending: `db.students.find().sort({average: 1})`

Descending: `db.students.find().sort({average: -1})`

5. Visualization

Bar chart: average marks per grade

Pie chart: department distribution

6. Data Flow

CSV → MongoDB → Flask API → Frontend → Table/Charts

7. Why MongoDB?

Flexible schema, NoSQL, scalable, aggregation support.

8. Concepts Used

CRUD, Aggregation, Sorting, Filtering, Pagination, REST APIs, Visualization.

9. Demo Explanation

Explain data ingestion, aggregation usage, and frontend-backend interaction clearly.